31. Mutexes, Atomics, and sync Primitives

31. Mutexes, Atomics, and sync Primitives

Phase: Phase 5 — Concurrency
Estimated time: 1–2 days
Slug: `31-mutexes-atomics`

Learning objectives

- Protect shared memory with mutexes
- Use RWMutex for read-heavy maps
- Know atomic operations for simple counters
- Prefer confining data to one goroutine when possible

Prerequisites

Complete prior numbered lessons. Type every example yourself.

Mutex

```
type Safe struct {
    mu sync.Mutex
    m  map[string]int
}

func (s *Safe) Inc(k string) {
    s.mu.Lock()
    defer s.mu.Unlock()
    s.m[k]++
}
```

Atomics

`atomic.Int64` for counters without mutex — not a free pass for complex invariants.

Design preference

1. Don't share (owning goroutine)
2. Channels for ownership transfer
3. Mutex for shared structures
4. Atomics for simple stats

Always `go test -race`.

Common mistakes (especially from Python instincts)

- Copying a struct containing a mutex
- Holding locks across slow I/O
- Assuming map access is safe concurrently

Exercises

Work in `exercises/31-mutexes-atomics/`. Read that folder's `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/31-mutexes-atomics/` until you have an honest failing attempt.

Self-check

1. Why defer Unlock?
2. RWMutex when?
3. Why race detector with mutex code?

Further reading

- https://pkg.go.dev/sync
- https://pkg.go.dev/sync/atomic